



UNIVERSITÄT
LEIPZIG

7. Vorlesung Support Vector Machine

INTERACTIVE VISUAL DATA MINING

Übersicht

7. Support Vektor Maschine

7.1 SVM

7.2 Soft-Margin SVM

7.3 Kernel Trick

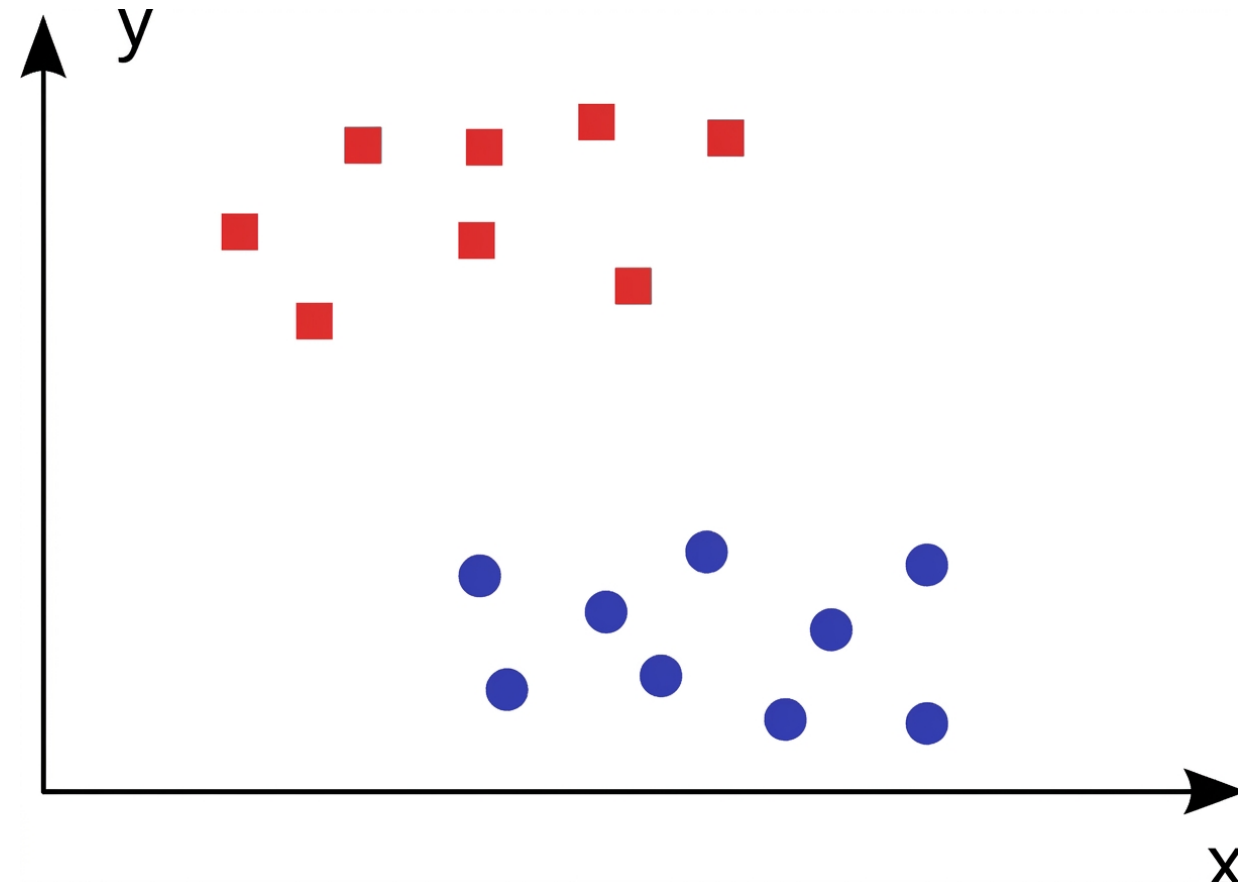
7.4 Multiclass SVM

Lernziele:

- Funktionsweise einer SVM als Klassifikator erklären
- Rolle von Hyperebene und Stützvektoren beschreiben und den Zusammenhang zwischen Margin und Generalisierung verstehen
- Einfluss des Parameters C auf Fehlertoleranz und Modellverhalten beschreiben bei der Soft-Margin SVM
- Kernel Trick erklären
- Verstehen wie SVMs für Multiclass Probleme erweitert werden können

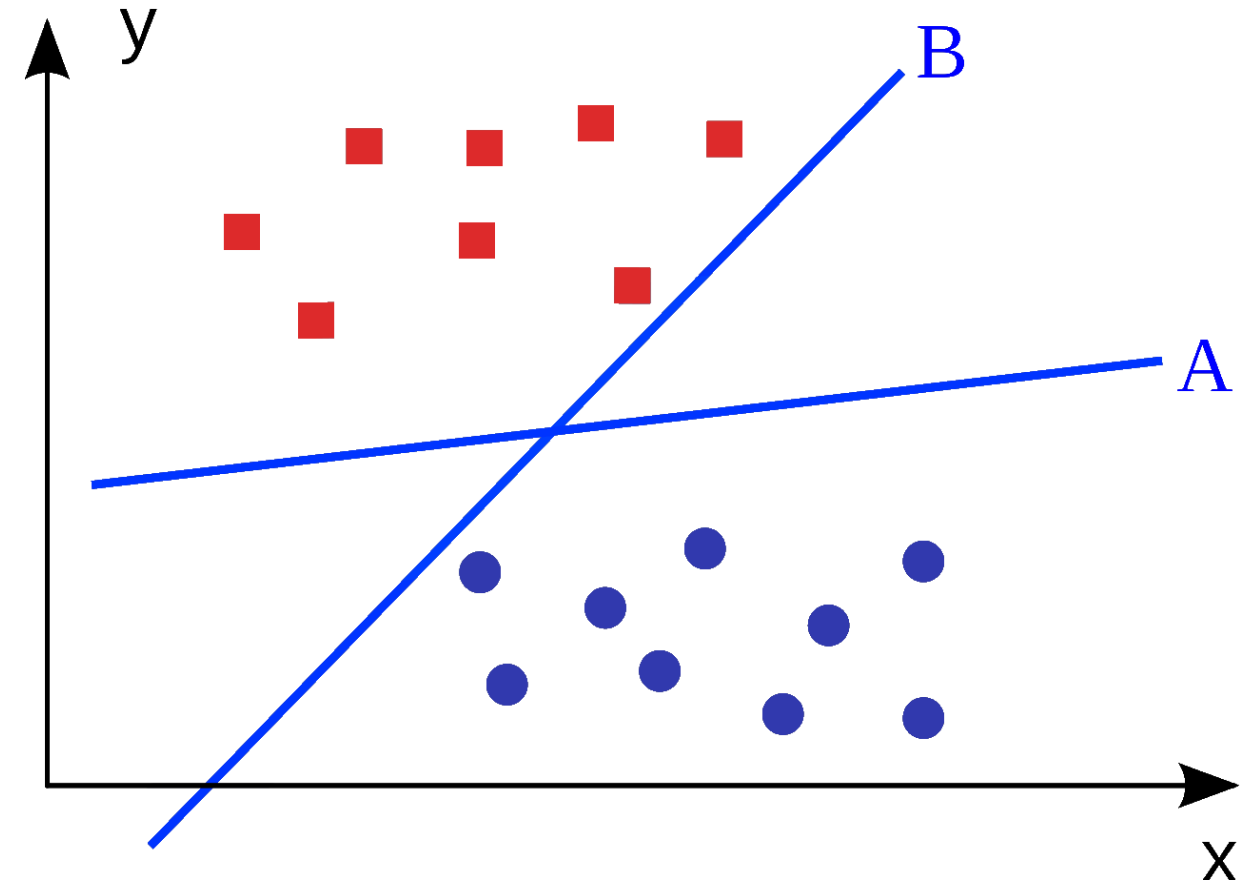
SUPPORT VECTOR MACHINE

- Klassifikationsmethode, welche auf Stützvektoren basiert
- Teilt dabei den Raum mit einer Hyperebene



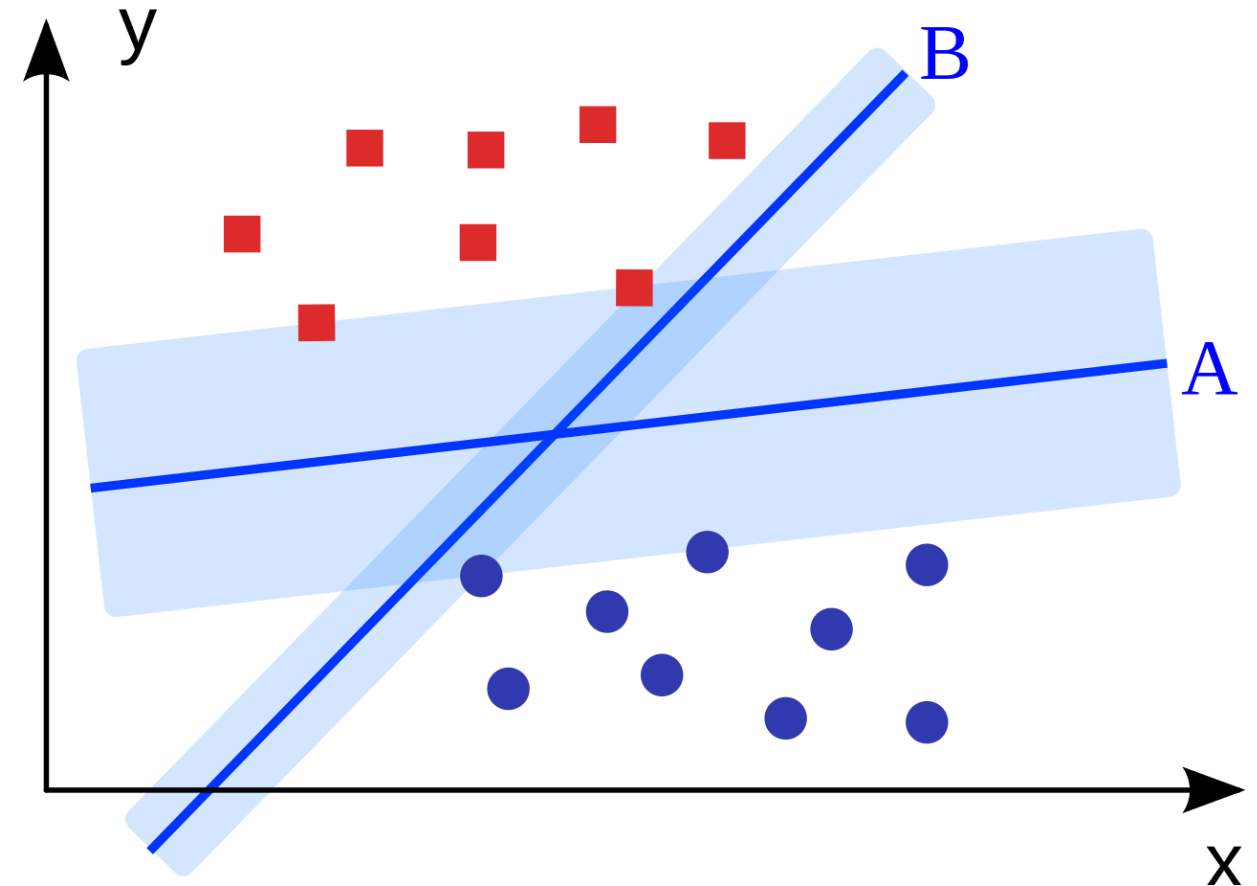
SUPPORT VECTOR MACHINE

- Klassifikationsmethode, welche auf Stützvektoren basiert
- Teilt dabei den Raum mit einer Hyperebene
- Aber: Multiple Hyperebenen sind gleich gut auf den Trainingsdaten



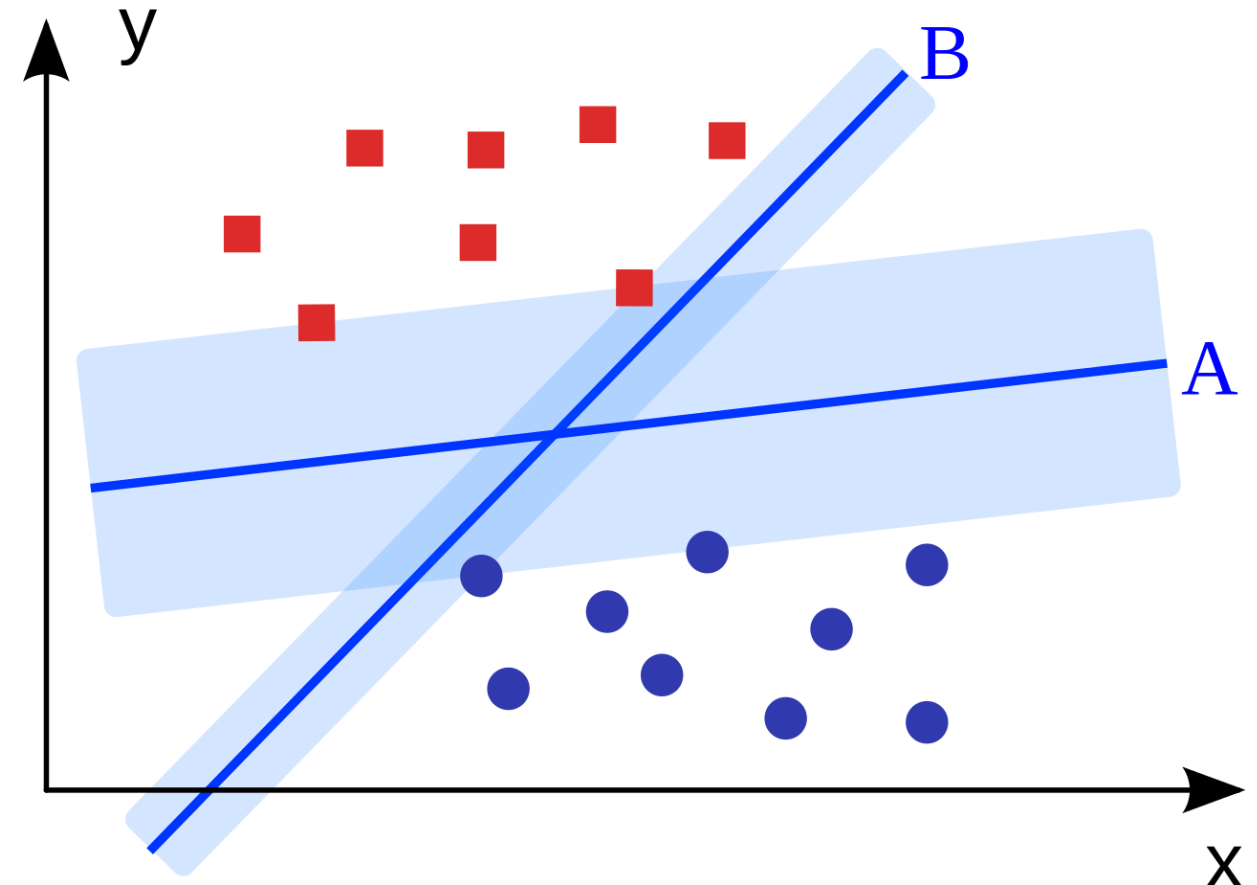
SUPPORT VECTOR MACHINE

- Klassifikationsmethode, welche auf Stützvektoren basiert
- Teilt dabei den Raum mit einer Hyperebene
- Aber: Multiple Hyperebenen sind gleich gut auf den Trainingsdaten
- → Suche nach der Hyperebene mit der größten Breite
- Maximum margin method



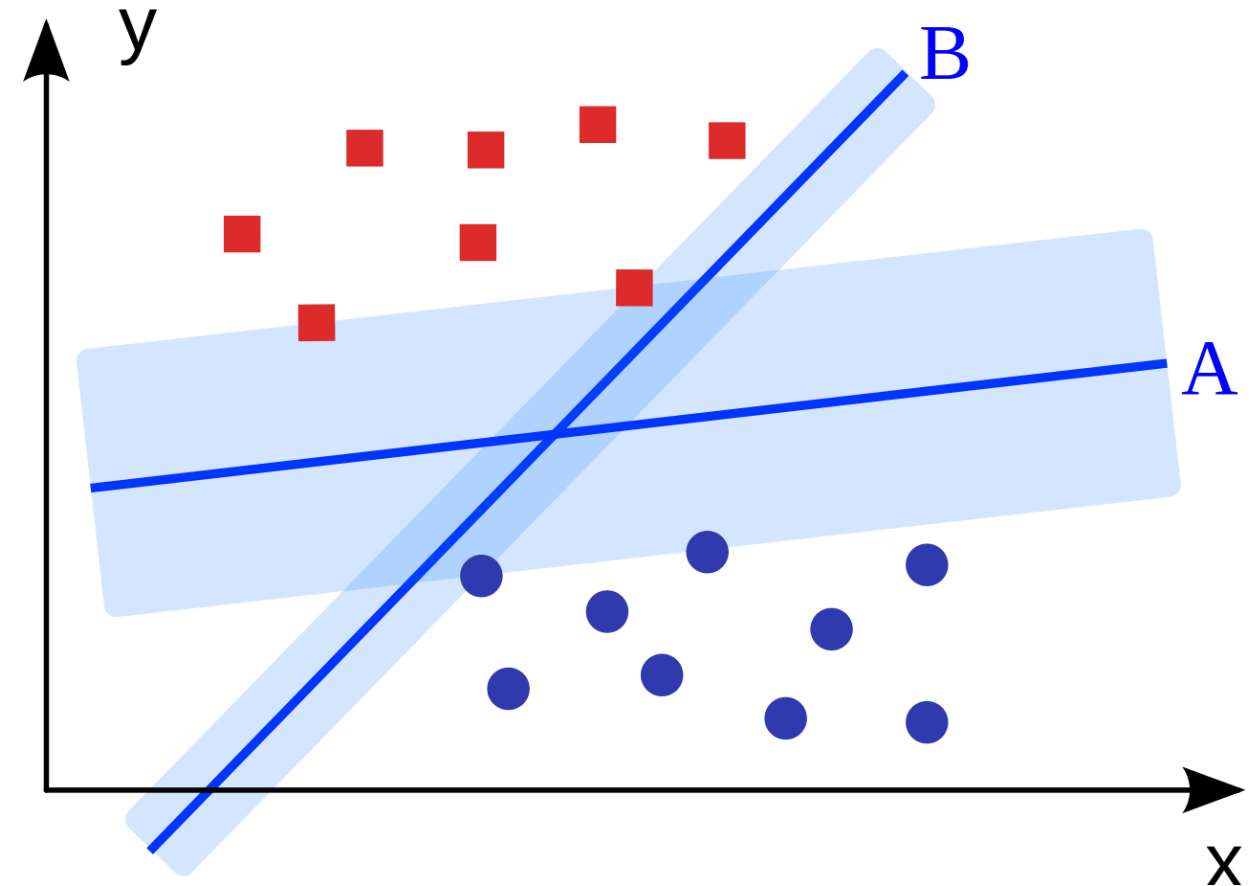
SUPPORT VECTOR MACHINE

- Eine SVM ist also im simplen Fall ein linearer Klassifikator
- Benötigt linear separierbare Klassen
- Meistens aber nicht vorhanden!
- Nutzung des sog. „Kernel Tricks“
- Warum Optimierung zum breitesten Rand?

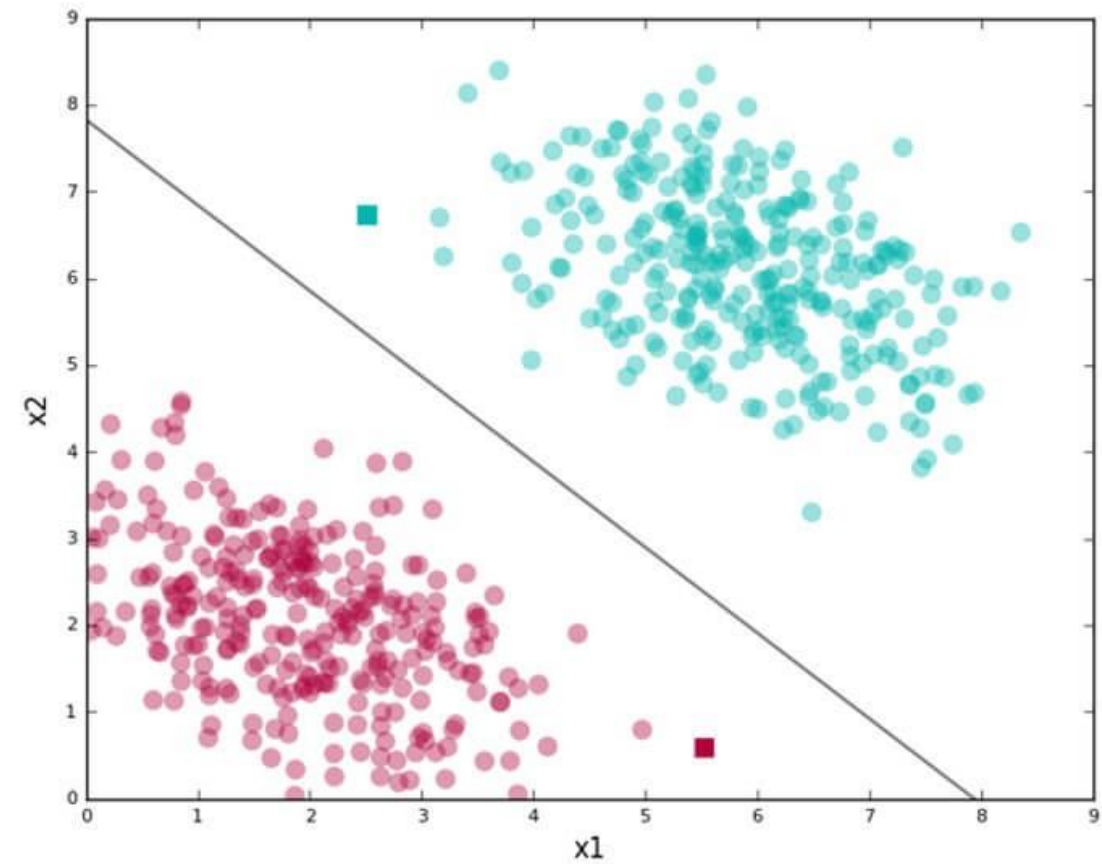
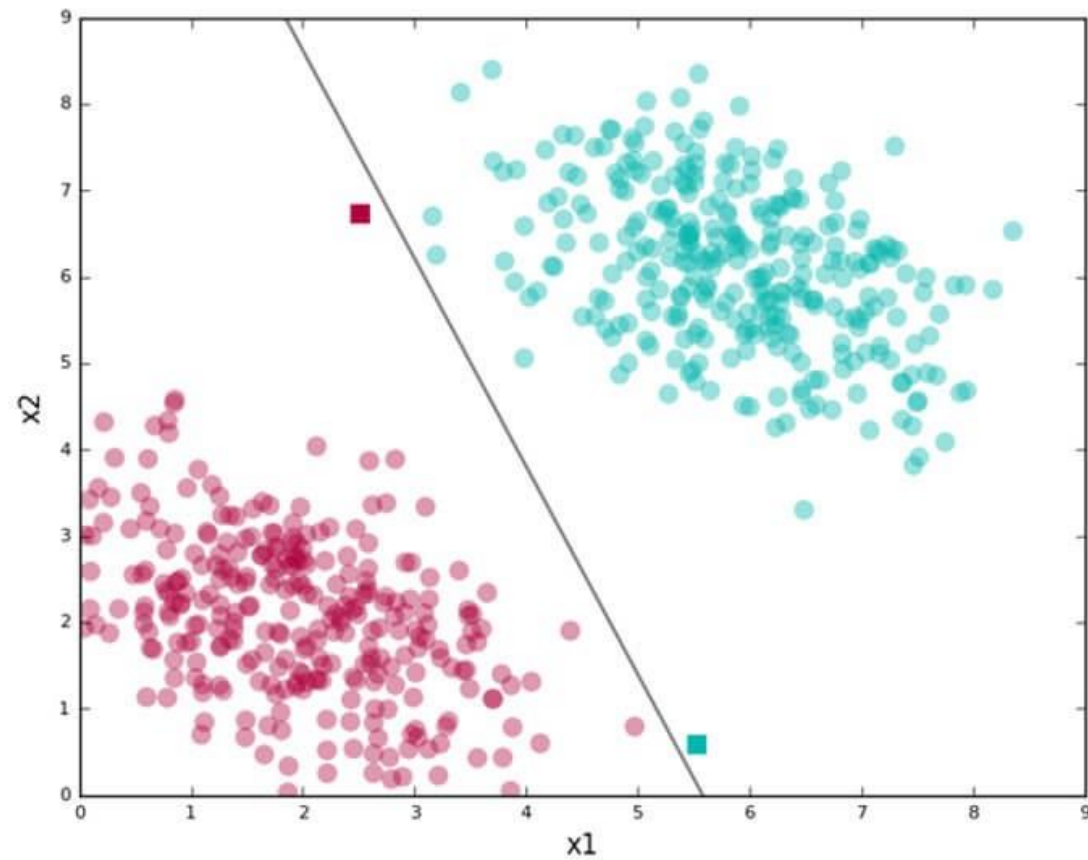


SUPPORT VECTOR MACHINE

- Eine SVM ist also im simplen Fall ein linearer Klassifikator
- Benötigt linear separierbare Klassen
- Meistens aber nicht vorhanden!
- Nutzung des sog. „Kernel Tricks“
- Warum Optimierung zum breitesten Rand?
- Objekte, die leicht abweichen, werden trotzdem richtig klassifiziert

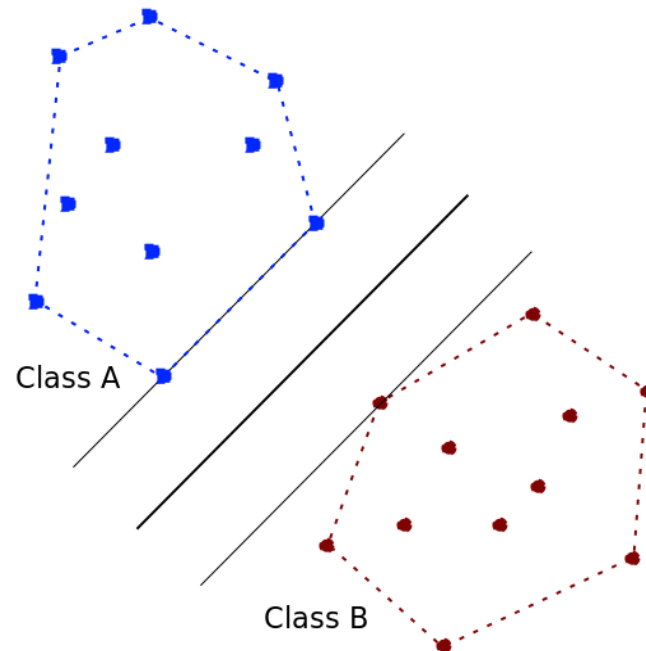


SUPPORT VECTOR MACHINE



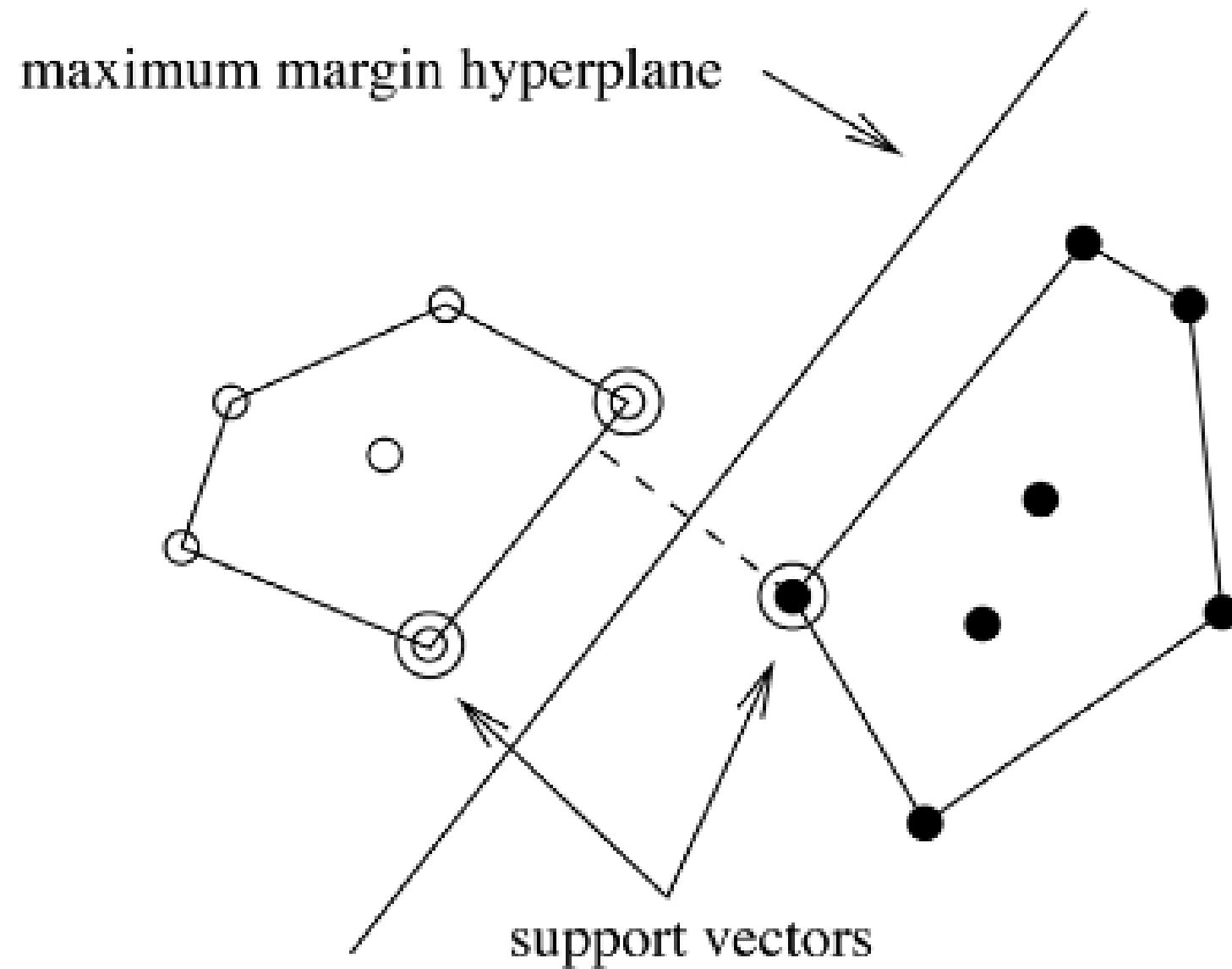
SUPPORT VECTOR MACHINE

- Genereller Ablauf:
 - Identifikation der sog. Stützvektoren
 - Datenpunkte, die nah an der zu findenden Hyperebene liegen
 - Datenpunkte, die am Rand ihrer Klasse liegen
 - Nutzung der konvexen Hülle

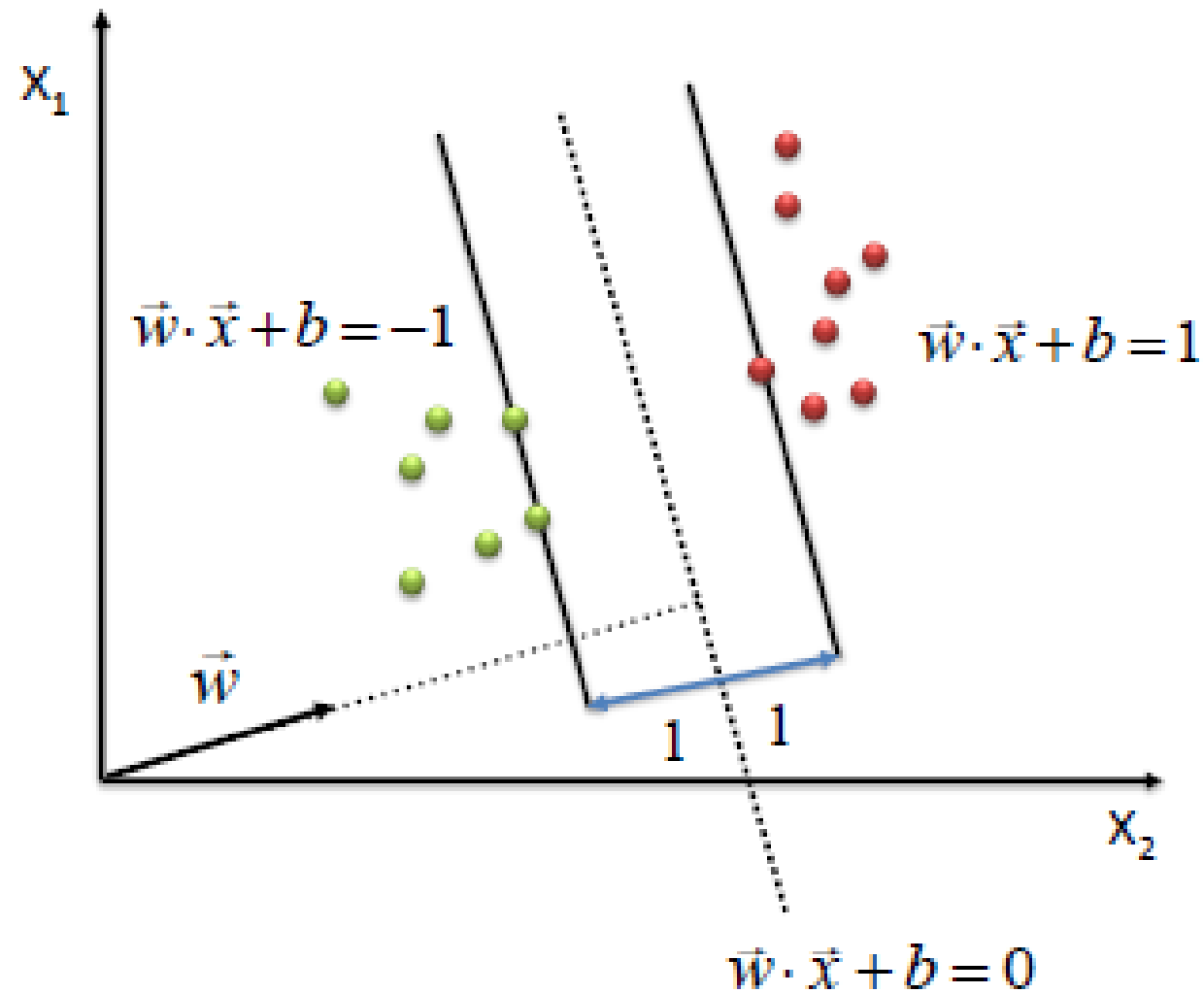


- Berechnung der Datenpunkte auf den konvexen Hüllen, die möglichst nah beieinander liegen
- Es gibt immer mindestens einen Stützvektor pro Klassen im Datensatz, meistens mehrere
- Was passiert, wenn die konvexen Hüllen der Daten überlappen?

SUPPORT VECTOR MACHINE



SUPPORT VECTOR MACHINE



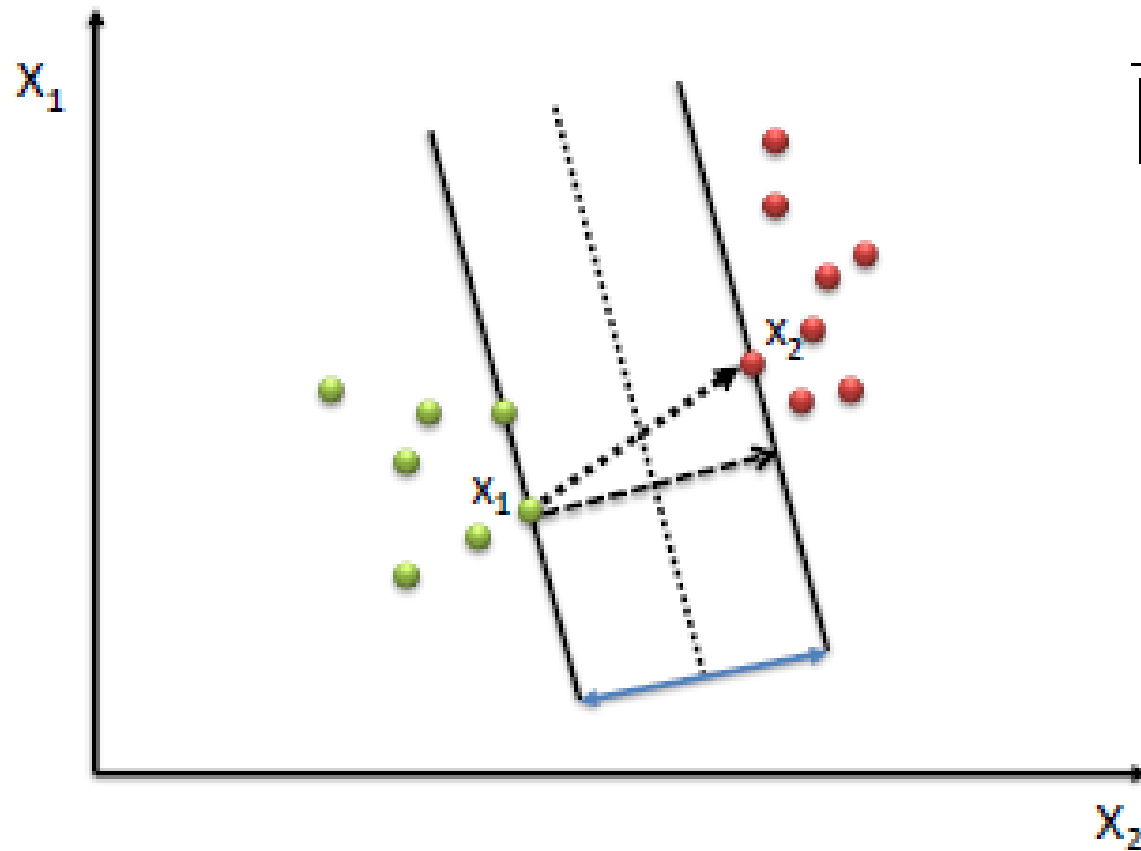
$$\max \frac{2}{\|\vec{w}\|}$$

s.t.

$$(w \cdot x + b) \geq 1, \forall x \text{ of class 1}$$

$$(w \cdot x + b) \leq -1, \forall x \text{ of class 2}$$

SUPPORT VECTOR MACHINE



$$\frac{w}{\|w\|} \cdot (x_2 - x_1) = \text{width} = \frac{2}{\|w\|}$$

$$w \cdot x_2 + b = 1$$

$$w \cdot x_1 + b = -1$$

$$w \cdot x_2 + b - w \cdot x_1 - b = 1 - (-1)$$

$$w \cdot x_2 - w \cdot x_1 = 2$$

$$\frac{w}{\|w\|} (x_2 - x_1) = \frac{2}{\|w\|}$$

SUPPORT VECTOR MACHINE

- Berechnung von b und w nun mittels einer Optimierungsfunktion
 - Sie ist quadratisch
 - Es existiert genau ein (globales) Minimum
 - Sie ist eingeschränkt (nicht jede Wahl von w und b trennt die Klassen)
 - N Einschränkungen (für jeden Punkt)
 - Berechnung mittels Lagrange Multiplikatoren
 - α_i „bestrafen“ Verletzungen der Einschränkungen
 - Nur Punkte mit $\alpha_i > 0$ sind Support-Vektoren alle anderen haben später $\alpha_i = 0$

$$\min \frac{1}{2} \|\mathbf{w}\|^2$$

$$s.t. y_i (\mathbf{w} \cdot \mathbf{x}_i + b) \geq 1, \quad \forall \mathbf{x}_i$$

$$\mathcal{L}(\mathbf{w}, b, \boldsymbol{\alpha}) = \frac{1}{2} \|\mathbf{w}\|^2 - \sum_{i=1}^N \alpha_i [y_i (\mathbf{w}^\top \mathbf{x}_i + b) - 1]$$

SUPPORT VECTOR MACHINE

- Ableitung der Lagrangefunktion nach w wird gleich 0 gesetzt (globales Minimum)
- Der Normalenvektor w kann dann als Linearkombination der Trainingsbeispiele dargestellt werden

$$w = \sum_{i=1}^N \alpha_i y_i x_i.$$

- Damit kann man die Klassifikationsregel für einen Datenpunkt x besser beschreiben durch Ersetzen von w

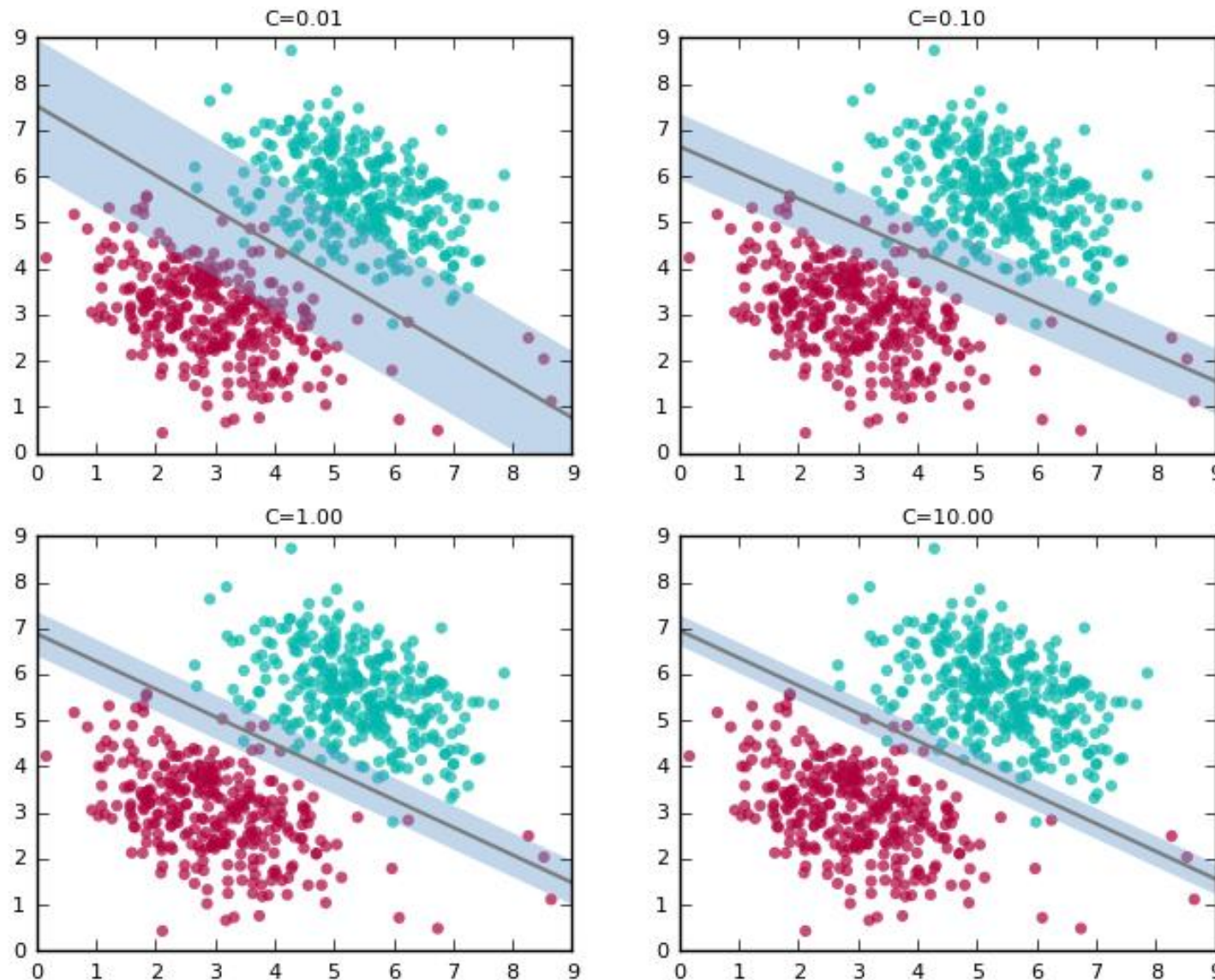
$$f(x) = \text{sgn}\left(\sum_{i=1}^N \alpha_i y_i \langle x, x_i \rangle + b\right).$$

- b und α_i sind die erlernten Parameter
- y_i ist die Klasse des Stützvektors x_i
- x ist der Vektor einer Testinstanz
- Berechnung des Skalarprodukts zwischen x_i und x

SUPPORT VECTOR MACHINE

- Soft-Margin SVM mit Fehlergrad C
 - Eine maximale Margin unter vollständiger Fehlerfreiheit kann zu schlechter Generalisierung führen (Overfitting)
 - Nutzung eines erlaubten Fehlergrads C
 - Dieser definiert den Tradeoff zwischen Korrektheit auf den Trainingsdaten und dem Freiheitsgrad für die Hyperebene
- Daten müssen generell linear trennbar sein
- Nah beieinanderliegende Daten können so besser getrennt werden
- Fehler in den annotierten Trainingsdaten haben einen geringeren Einfluss

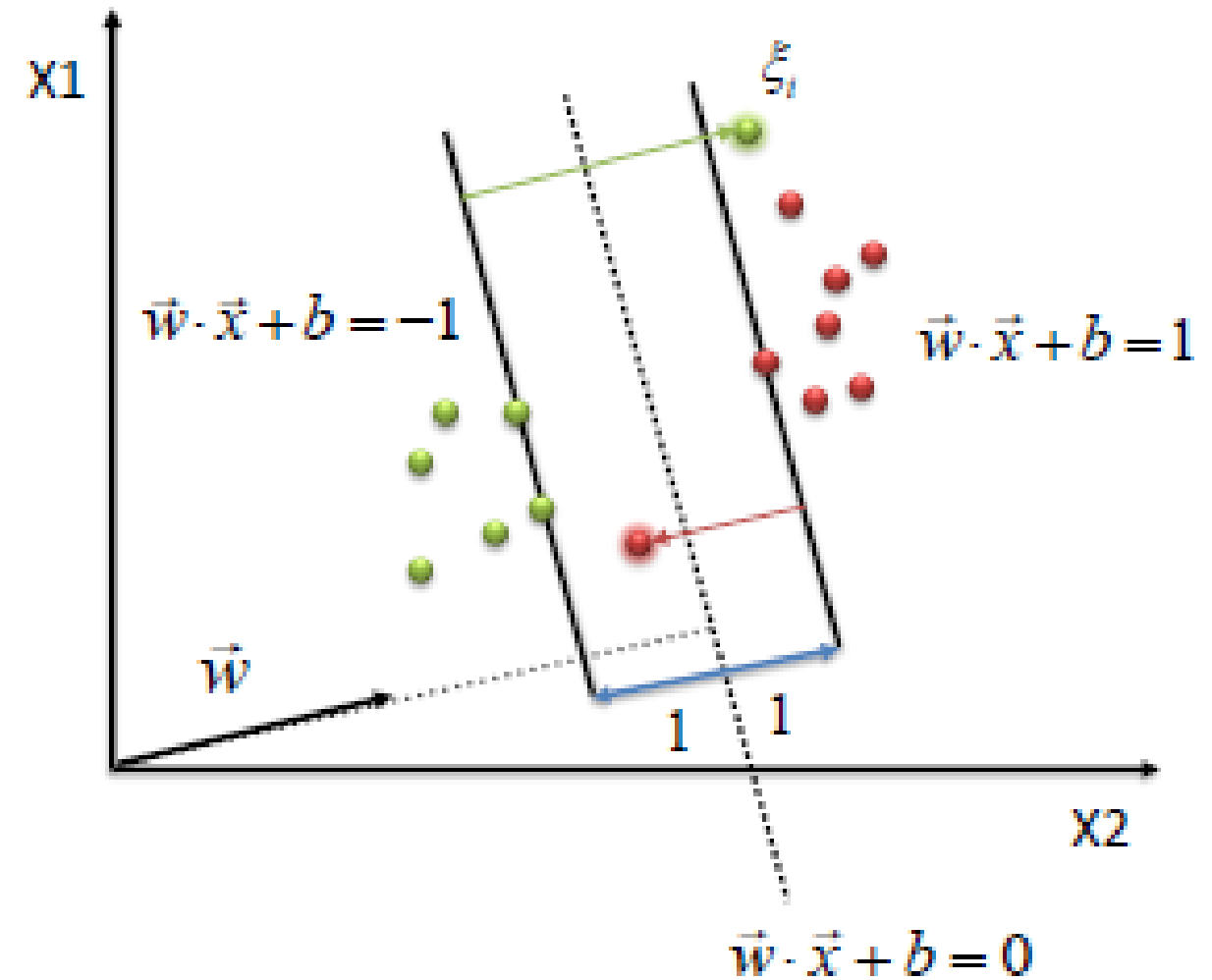
SUPPORT VECTOR MACHINE



- Niedriges C : Fehler werden weniger bestraft → Modell toleriert Fehlklassifikationen, größere Margin
- Hohes C : Fehler werden stark bestraft → Modell versucht Klassen möglichst exakt zu trennen, kleine Margin
- $C \rightarrow \infty$: Hard Margin, Keine Fehler möglich, eventuell nicht lösbar

SUPPORT VECTOR MACHINE

- Soft-Margin mit Fehlergrad C



SUPPORT VECTOR MACHINE

- Soft-Margin mit Fehlergrad C
 - C ist der Gewichtungsfaktor (Hyperparameter)
 - Großes $C \rightarrow$ Fehler sehr stark bestraft $\rightarrow$ fast harte Margin
 - Kleines $C \rightarrow$ mehr Fehlertoleranz $\rightarrow$ weichere Margin
 - Minimiert die Summe der Margin-Verletzungen (Slack-Variablen), während die Margin maximiert wird.
 - Optimiert dabei **nicht** die Anzahl der Fehlklassifikationen $\rightarrow$ Dies wäre NP-hart

- Anpassung der Einschränkung

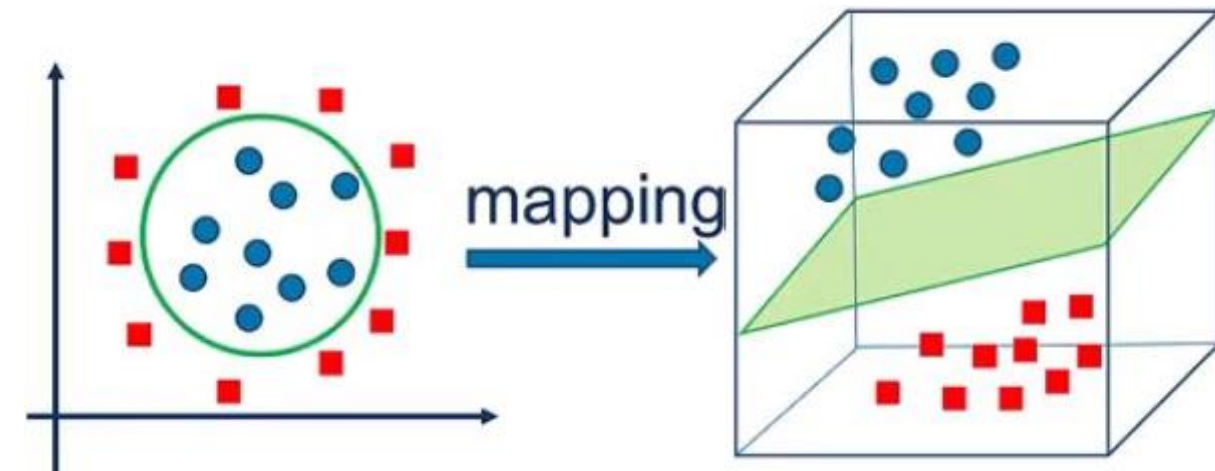
$$y_i(w \cdot x_i + b) \geq 1 - \xi_i, \quad \forall x_i$$
$$\xi_i \geq 0$$

- Anpassung der Optimierungsfunktion

$$\min \frac{1}{2} \|w\|^2 + C \sum_i \xi_i$$

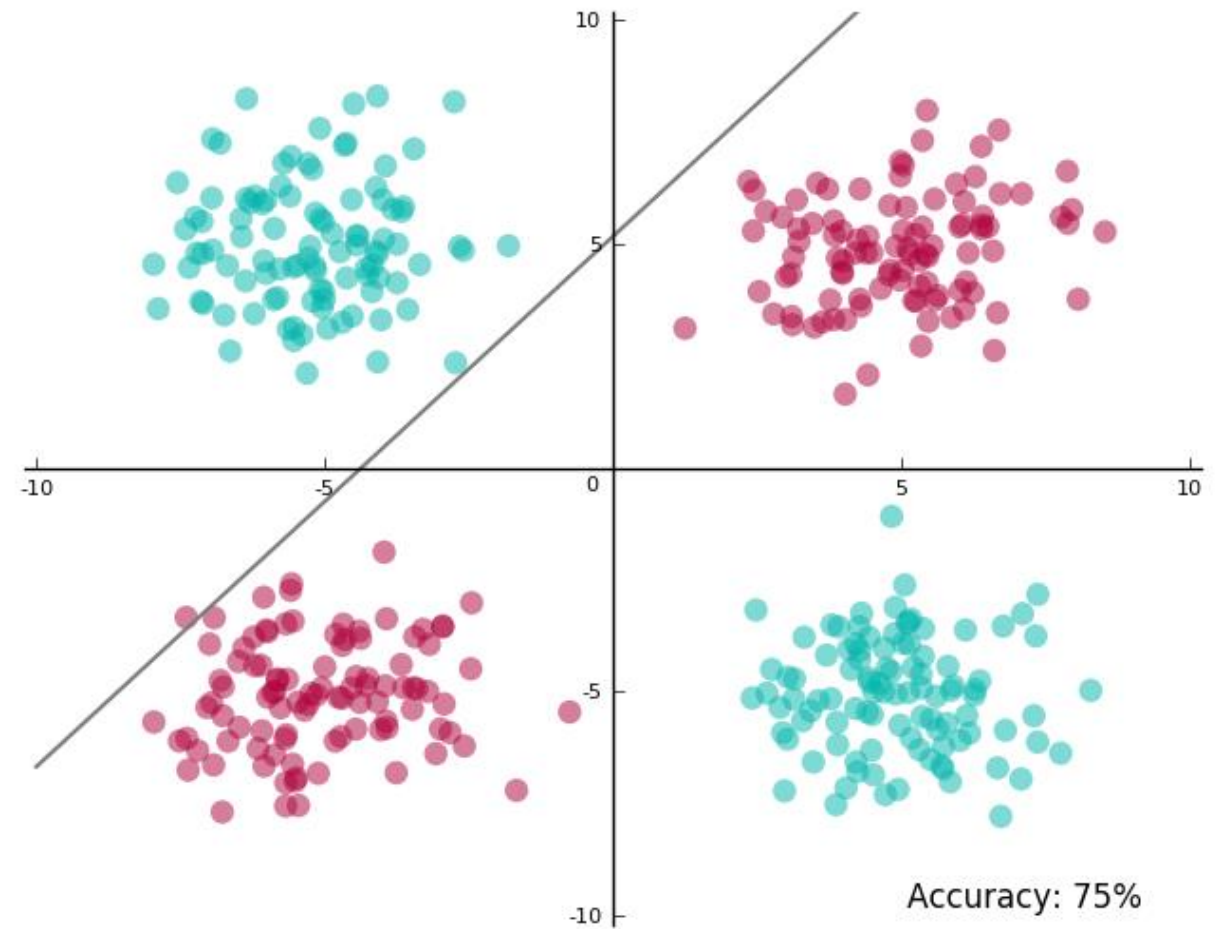
SUPPORT VECTOR MACHINE

- Nicht linear trennbare Daten
- SVMs können nur lineare trennbare Daten klassifizieren!
- In der Realität sind die Datensätze aber meistens nicht linear trennbar
- → Nutzung des „Kernel Tricks“



SUPPORT VECTOR MACHINE

- Nicht linear trennbare Daten
- Transformiere den Datensatz mittels einer nicht linearen Kernel Funktion



SUPPORT VECTOR MACHINE

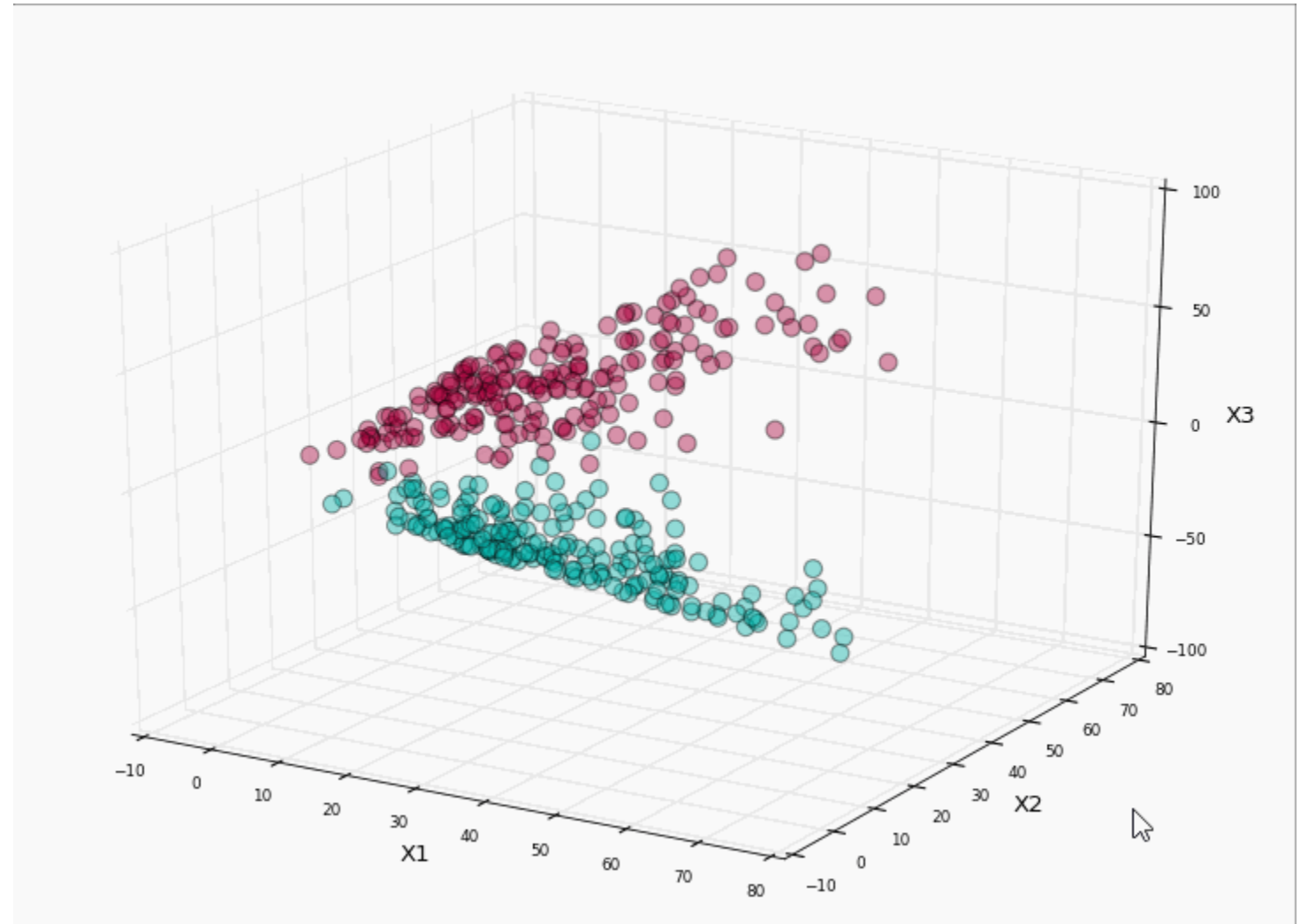
- Nicht linear trennbare Daten
- Transformiere den Datensatz mittels einer nicht linearen Kernel Funktion
- Beispiel quadratischer Kernel:

$$\begin{aligned}K(x, y) &= (x^\top y)^2 = (x_1 y_1 + x_2 y_2)^2 \\ &= x_1^2 y_1^2 + 2x_1 x_2 y_1 y_2 + x_2^2 y_2^2\end{aligned}$$

$$X_1 = x_1^2$$

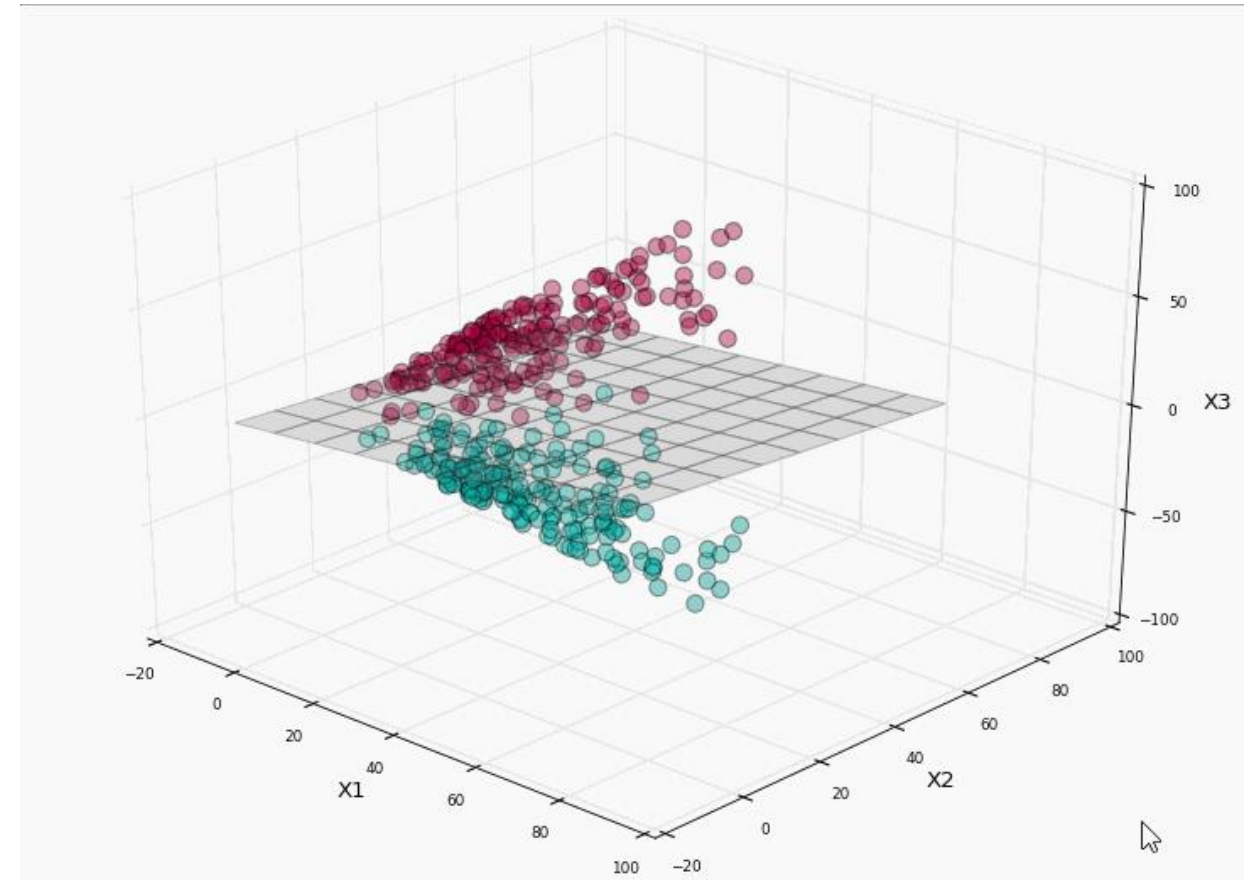
$$X_2 = x_2^2$$

$$X_3 = \sqrt{2}x_1 x_2$$



SUPPORT VECTOR MACHINE

- Nicht linear trennbare Daten
- Transformiere den Datensatz mittels einer nicht linearen Kernel Funktion
- Nach der Transformation berechnet man die Hyperebene im hochdimensionalen aber linear trennbaren Raum
- Testinstanzen werden auch transformiert und dann klassifiziert



SUPPORT VECTOR MACHINE

- Nicht linear trennbare Daten

- Es gibt verschiedene Kernel zum Transformieren

- Homogen Polynomial

$$K(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^\top \mathbf{y})^d$$

- Inhomogen Polynomial

$$K(\mathbf{x}, \mathbf{y}) = (\mathbf{x}^\top \mathbf{y} + c)^d$$

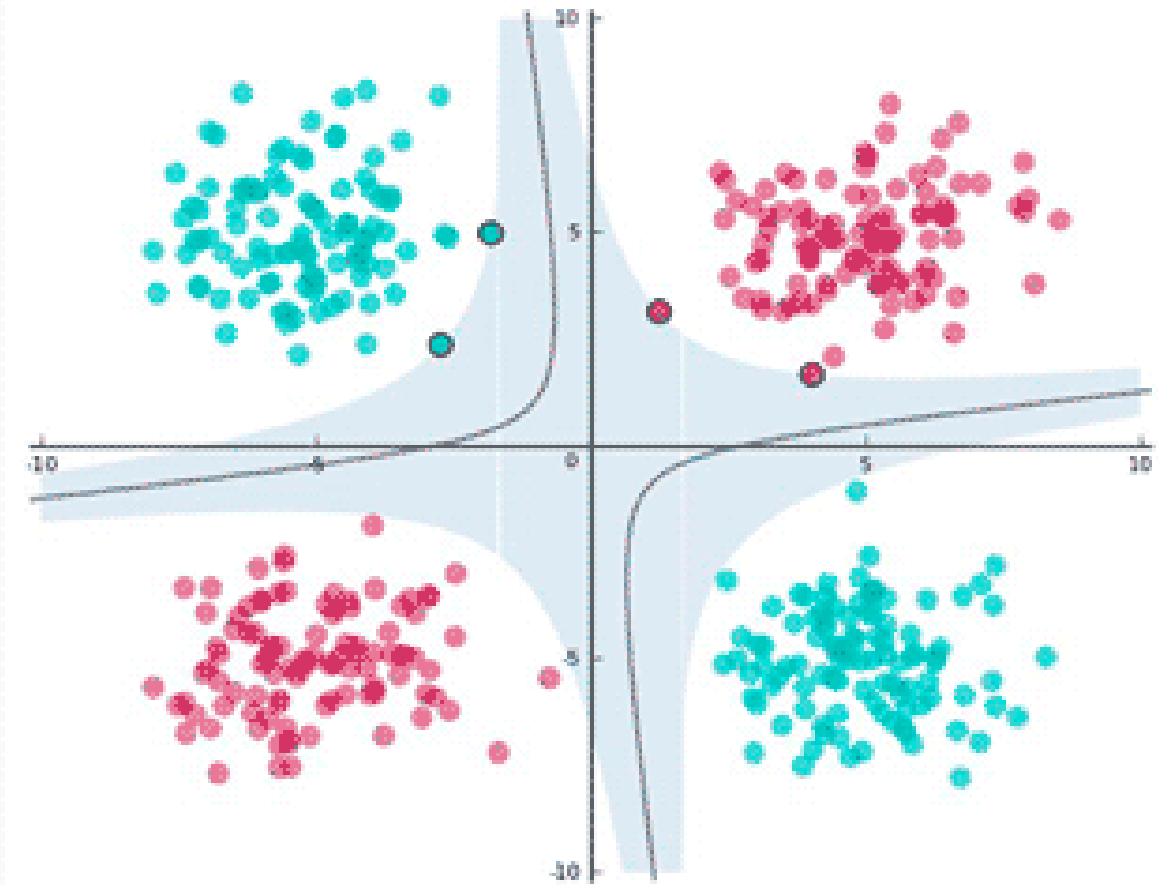
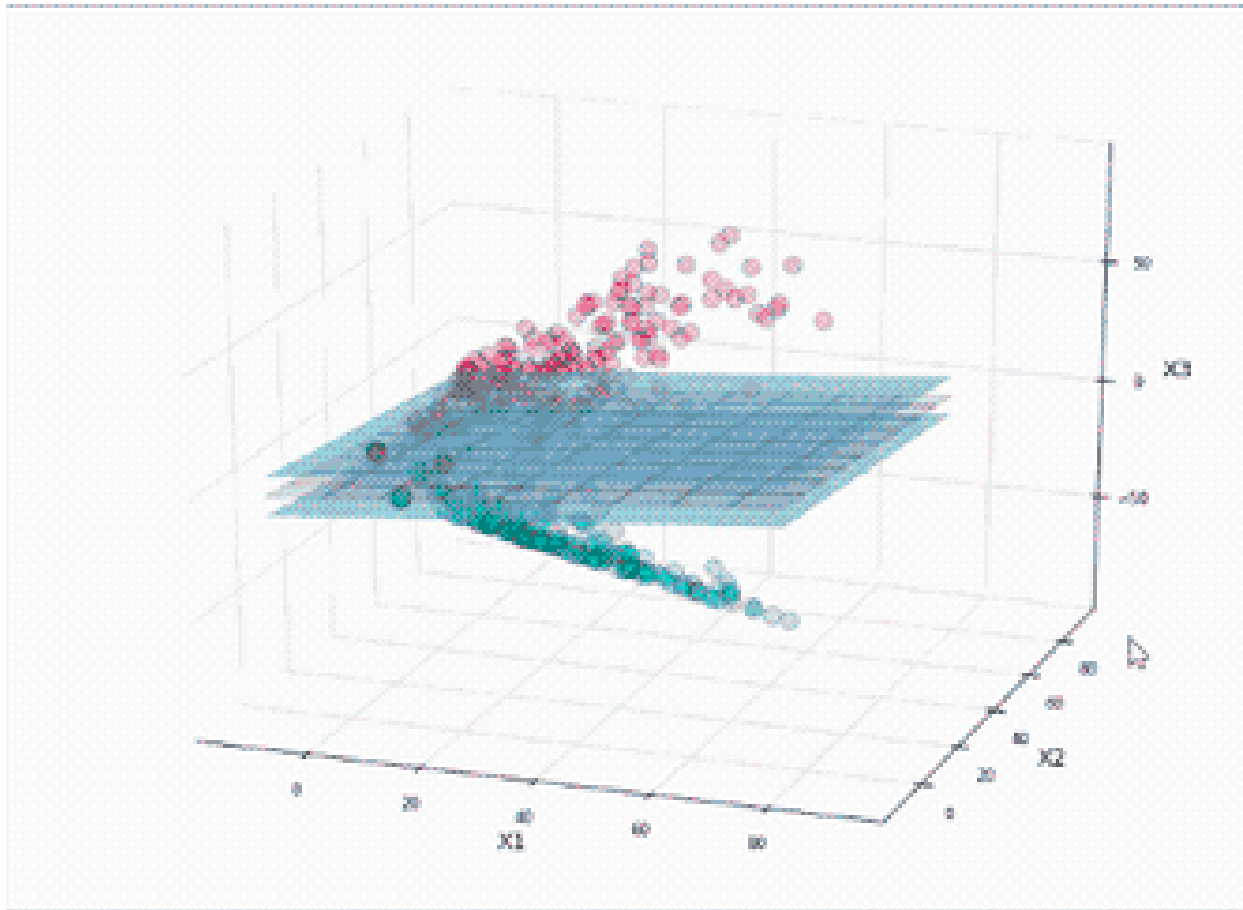
- Gaußsche radiale Basisfunktion

$$K(\mathbf{x}, \mathbf{y}) = \exp \left(-\gamma \|\mathbf{x} - \mathbf{y}\|^2 \right)$$

- $\gamma > 0$ = Parameter, der die Breite der Gauß-Kurve steuert
- Kleines $\gamma \rightarrow$ breite Glocke / weiche Trennung (wenig Einfluss von einzelnen Punkten)
- Großes $\gamma \rightarrow$ schmale Glocke / scharfe Trennung, mehr Risiko von Overfitting
- Starte mit einem niedrigen γ und erhöhe bis die Genauigkeit des Klassifikators steigt

SUPPORT VECTOR MACHINE

- Nicht linear trennbare Daten



SUPPORT VECTOR MACHINE

- Nicht linear trennbare Daten
- Problem: Pseudodimensionen lassen den Rechenaufwand enorm ansteigen!
 - Skalarprodukt bei der Optimierung und Klassifikation hängt von der Anzahl der Dimensionen ab

- Beispiel:

- Ursprung: $\vec{x}_i = (x_{i1}, x_{i2})$

- Transformiert:

$$\vec{X}_i = (x_{i1}^2, x_{i2}^2, \sqrt{2}x_{i1}x_{i2})$$

- Benötigt 8 Multiplikation für die Transformation, 3 im Skalarprodukt sowie 2 Additionen → 13 Operationen

$$K(\vec{x}_i, \vec{x}_j) = (\vec{x}_i \cdot \vec{x}_j)^2$$

SUPPORT VECTOR MACHINE

- Nicht linear trennbare Daten

- Es geht aber besser:

$$K(\vec{x}_i, \vec{x}_j) = (\vec{x}_i \cdot \vec{x}_j)^2$$

- Man berechnet das Skalarprodukt im Ursprungsraum und potenziert dieses mit der Anzahl der Dimensionen des transformierten Raums

- Damit benötigt man für das Beispiel nur noch 2 Multiplikationen für das Skalarprodukt und eine weitere für die Quadrierung
- Eine Addition im Skalarprodukt

→ 4 Operationen

$$K(\vec{x}_i, \vec{x}_j) = (\vec{x}_i \cdot \vec{x}_j)^2 \tag{1}$$

$$= (x_{i1}x_{j1} + x_{i2}x_{j2})^2 \tag{2}$$

$$= x_{i1}^2x_{j1}^2 + x_{i2}^2x_{j2}^2 + 2x_{i1}x_{i2}x_{j1}x_{j2} \tag{3}$$

$$= (x_{i1}^2, x_{i2}^2, \sqrt{2}x_{i1}x_{i2}) \cdot (x_{j1}^2, x_{j2}^2, \sqrt{2}x_{j1}x_{j2}) \tag{4}$$

SUPPORT VECTOR MACHINE

- Feature Importance Analyse mit SVM
 - Bei einer linearen SVM mit standardisierten Daten gilt für die Gewichte:
 - Gewichte zeigen, welche Features die Entscheidung am meisten beeinflussen
 - große positive/negative Werte
→ stark einflussreiche Features.
Positiv Klasse A, Negativ Klasse B
 - nahe 0 → kaum relevant
- Standardisiert:
 - Mittelwert = 0, Varianz = 1
 - alle Features sind auf vergleichbarer Skala
 - Dann gilt: Gewichte sind direkt vergleichbar
- Nach Kernel Trick nicht mehr unbedingt anwendbar da Daten im verzerrten höherdimensionalen Raum liegen.

SUPPORT VECTOR MACHINE

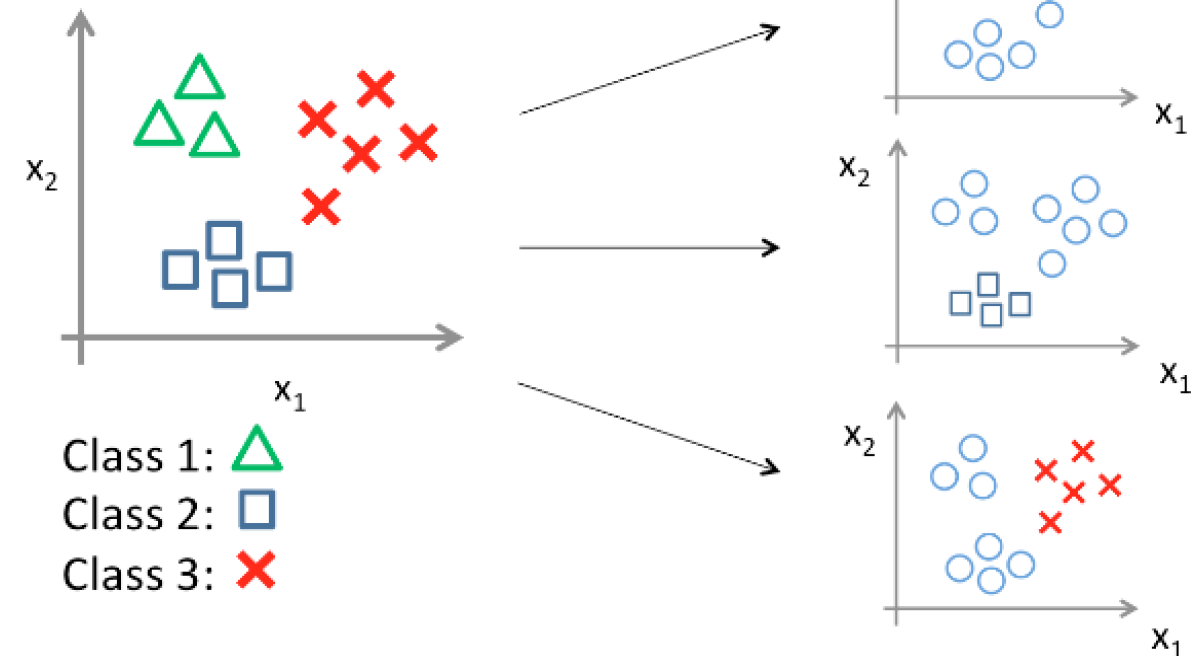
– Zusammenfassung

- SVM optimiert die trennende Hyperebene zwischen den Klassen
 - Ergebnis ist stabil, da das Optimierungsproblem nur ein globales Minimum hat
 - Kernel Trick erlaubt die Klassifikation nicht linearer Daten
 - Skalarproduktberechnung kann im Ursprungsraum durchgeführt werden
 - Beschränkung auf Stützvektoren vereinfacht die Optimierung
 - Veränderungen der Trainingsdaten führen nur bei neuen Stützvektoren zu anderen Klassifikatoren
- weniger anfällig auf Outlier

SUPPORT VECTOR MACHINE

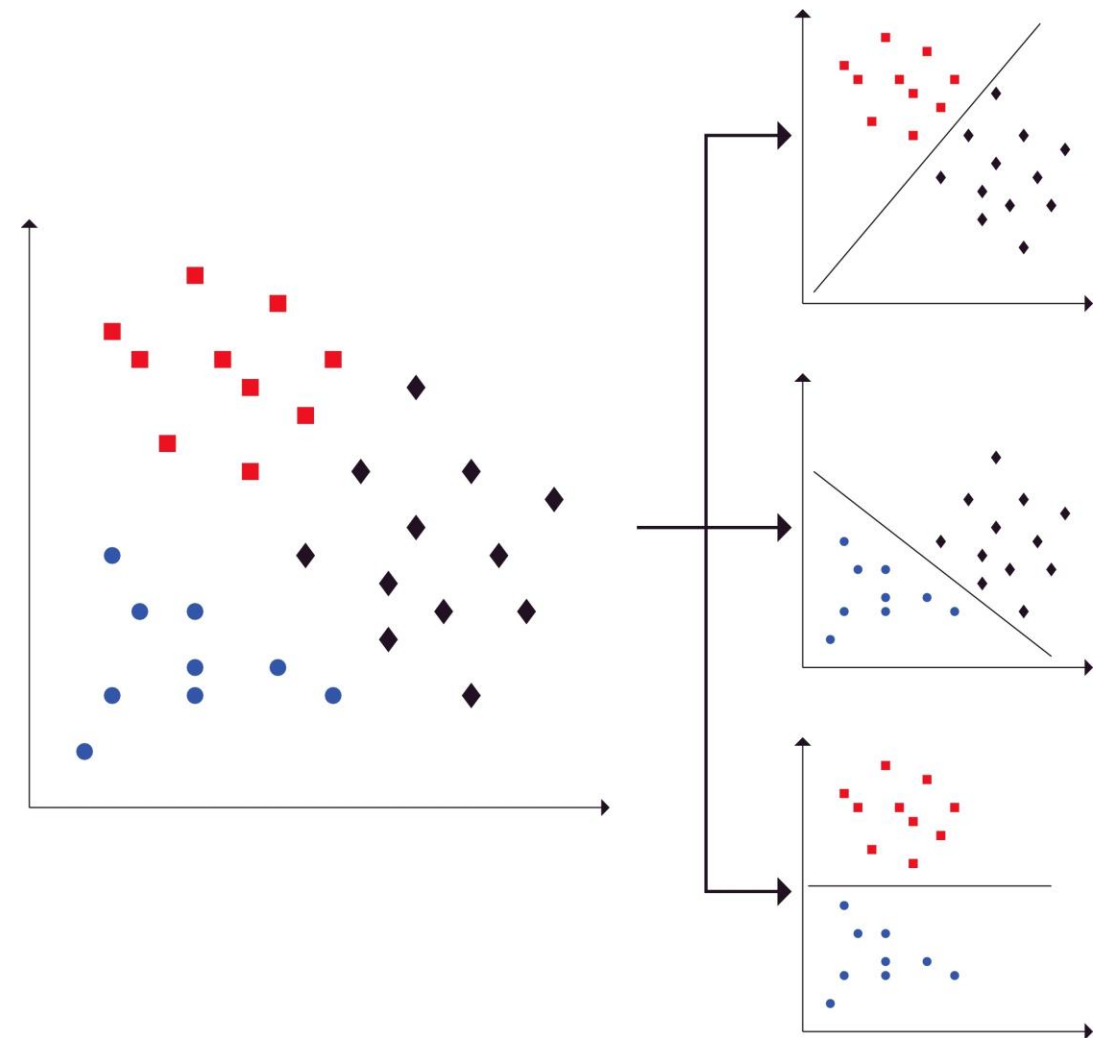
- Multiclass Fall mit k Klassen
- One-Against-All
- Benötigt nur k Klassifizierer, aber jeder arbeitet auf allen Daten
- Jeder Klassifizierer gibt einen Score aus (Abstand zur Hyperebene).
- Die Klasse mit dem höchsten Score wird gewählt.

One-vs-all (one-vs-rest):



SUPPORT VECTOR MACHINE

- Multiclass Fall mit k Klassen
 - One-Against-One
 - Lösen aller Binärprobleme benötigt $\frac{k(k-1)}{2}$ Klassifizierer, aber nutzt jedesmal nur eine Teilmenge der Daten
 - Jeder Klassifizierer entscheidet sich für eine von zwei Klassen.
 - Die Klasse mit den meisten Stimmen wird gewählt.
 - Mit einem gerichteten azyklischen Graph (DAG-SVM) kann die Klassifikation auf $k - 1$ Vergleiche im Turniermodus reduziert werden.



SUPPORT VECTOR MACHINE

- Methode von Crammer & Singer (2001)
 - Direkte Multiclass-Formulierung mit pro Klasse einen eigenen Gewichtungsvektor w_k
- Optimierungsproblem wird so formuliert, dass ein Modell alle Klassen trennt
- Optimierung mit: Margin-Bedingungen für alle Klassen
- Gewichtsmatrix M mit k Gewichtungsvektoren w
- Wähle Klasse mit größtem Skalarprodukt zwischen Gewicht und Datenpunkt

